

오늘의 작업기록 — 2026년 6월 29일

mini-ledger 증분 5 — transfer 멱등성 키 + 백엔드 "사고법" 정립

(코드보다 "어디에 뭘 어떤 순서로 박는가"를 처음 언어화한 날)

한눈에

항목	내용
목표	증분 5 — transfer에 멱등성 키(중복 요청 방지)
결과	완료 ✓ — 같은 키 두 번 → 한 번만 반영(잔액 1400) 검증
커밋	add idempotency key to transfer (Bank+Main)
저장소	github.com/Taewoo-Won/mini-ledger (7개 .java)
오늘의 핵심	설계 판단 3개를 이유까지 스스로 맞춤 + 백엔드 사고법 정립
새 개념	멱등성 · Set(HashSet) · return으로 조기 종료
틀린 곳	import 점 오타(import. → import), usedkeys 대소문자 — 교정
학습 방식	검사 위치/스킵/저장 시점을 말로 먼저 확정 → 코드는 거의 자동. "사고법"을 질문으로 풀어냄

코드 양은 적었다(transfer 한 곳 + Set 필드). 근데 오늘의 본체는 두 개다 — ① 멱등성 3판단을 **이유까지** 맞춤 것, ② "문법은 아는데 *어디에 뭘 어떤 순서로* 박을지 모르겠다"는 막막함을 **5단계·그릇 3질문·2층 구조로 언어화**한 것. 후자가 앞으로 모든 증분에 쓰일 진짜 자산이다.

0. 출발점 / 오늘의 목표

- **이전까지**: 증분 1~4 완료 (Account → Bank/transfer → 커스텀 예외 → 데이터모델+감사로그).
- **오늘 목표**: 증분 5 — transfer에 멱등성 키. 후보 3개(멱등성/이력조회/정산) 중 멱등성으로 확정 (결제 핵심 + 6/24 cron 멱등성의 결제 맥락 완결).
- **다음**: 멱등성을 deposit/withdraw로 확장 or 이력 조회(계좌 필터). 다음 세션에 스코프.
- **작업 도구**: 폰 + VPS + nano + javac. (긴 복붙 금지·nano 직접·cat 확인 — 6/28 규칙 유지.)

PART 1 — 왜 멱등성인가 (도메인 동기)

상황: 토스에서 "1000원 이체" 눌렀는데 응답이 안 온다(타임아웃). 됐는지 안 됐는지 모름 → 재시도. 첫 요청이 이미 처리됐다면 → **두 번 이체, 돈 두 배**.

해법 — **멱등성 키**: 각 요청에 고유 키(신분증)를 붙인다. 서버는 키로 "이 요청 처음인가, 봤던 건가" 구분. 본 키면 다시 안 함 → **몇 번 재시도해도 결과는 한 번**.

★ 6/24 cron의 "변경 없으면 커밋 안 함"과 **정확히 같은 원리**. 같은 명령 1440번 = 결과 한 번. 그 개념이 이제 결제 맥락에서 완결됐다. Stripe·토스 모든 결제 API의 기본.

PART 2 — 설계 판단 3개 (오늘의 본체, 이유까지 스스로)

코드 짜기 전에 말로 먼저 확정할 3가지. **전부 이유까지 맞춤**.

판단	결정	이유 (내가 댄 것)
① 검사 위치	withdraw/deposit 위 (돈 움직이기 전)	"돈은 실수하면 안 되니 먼저 거른다." (증분 3: 검증은 상태 변경 전)
② 이미 있는 키면	조용히 스킵(return)	"중복이란 뜻이니 반영 안 함." void라 반환할 게 없으니 스킵이 자연스러움
③ 키 저장 시점	이체 성공 후 (맨 아래)	"실패한 거래 키를 저장하면 재시도가 영영 막힘 → 성공한 것만 저장"

★ 트레이드오프 이해: 진짜 결제 API는 ②에서 "이전 결과 반환"을 한다(재시도 클라가 원래 응답 받게). 근데 그건 메서드가 뭔가 반환할 때. 지금 transfer는 **void** 라 **스킵이 맞다**. 이 맥락 의존성을 알고 감.

→ 세 판단이 그대로 코드 구조가 됨: **검사 맨 위 → 이체 가운데 → 저장 맨 아래**.

PART 3 — Bank.transfer 멱등성 (내 코드 vs 정답)

3-1. Set 그릇 — 왜 Set인가

처리한 키 추적엔 순서·값 불필요, "이 키 봤나?"만 필요 → **Set**.

```
import java.util.Set;
import java.util.HashSet;
private final Set<String> usedKeys = new HashSet<>();
```

- `usedKeys.contains(key)` → 있나?(true/false) / `usedKeys.add(key)` → 추가
- 틀린 곳: `import java.util.Set;` (접) → `import java.util.Set;` (공백). `usedkeys` → `usedKeys` (대소문자 통일).

3-2. transfer — 최종

```
public void transfer(String fromId, String toId, long amount, String idempotencyKey) {
    if (usedKeys.contains(idempotencyKey)) {
        return; // ① 검사 맨 위 → 스킵
    }
    Account fromAccount = getAccount(fromId);
    Account toAccount = getAccount(toId);
    fromAccount.withdraw(amount);
    toAccount.deposit(amount);
    history.add(new Transaction(TransactionType.TRANSFER, fromId, toId, amount));
    usedKeys.add(idempotencyKey); // ③ 저장 맨 아래 (성공 후)
}
```

- 매개변수 4번째 `String idempotencyKey` 추가 → 시그니처 변경 → Main 호출도 다 고쳐야 함(아래).

3-3. return 의 의미 (새 개념)

- `void` 메서드에서 `return;` 은 "여기서 그냥 끝, 아래 줄 실행 안 함." 돌려줄 값 없이 빠져나감.
- 이미 본 키면 → 검사에서 return → withdraw/deposit 아예 안 달음 → 중복 이체 차단.

PART 4 — Main 수정 + 멍등성 검증

4-1. 시그니처 변경 → Main 컴파일 에러 (정상 신호)

Bank의 transfer를 바꾸자 Main에서 에러 3개:

```
required: String,String,long,String   (새 transfer)
found:   String,String,int             (옛 호출, 키 없음)
Main.java:6, 13, 19
```

→ 이게 "다음 고칠 곳"을 컴파일러가 짚어준 것. 에러 = 네비게이션.

4-2. Main 수정 — 호출에 키 추가 + 검증용 dup

```
bank.transfer("a", "b", 200, "dup-key"); // 검증용
bank.transfer("a", "b", 200, "dup-key"); // ← 같은 키 두 번!
bank.transfer("a", "b", 1000, "key-1");
...
bank.transfer("a", "z", 500, "key-2"); // try (없는 계좌)
bank.transfer("a", "b", 999999, "key-3"); // try (잔액 부족)
```

4-3. 실행 결과 — 멍등성 작동 증명

```
1400      ← a 잔액
4400      ← b 잔액
===== 거래 내역 =====
TRANSFER a -> b : 200      ← dup-key, 한 건만! (두 번 호출했는데)
TRANSFER a -> b : 1000
DEPOSIT null -> a : 500
WITHDRAW b -> null : 300
```

- `TRANSFER ... 200` 이 한 건만 = 두 번째 dup-key 스킵됨.
- a 잔액 **1400** 검증: $2100 - 200(\text{한 번만}) - 1000 + 500 = 1400$. (두 번 빠졌으면 1200.)
- → 멍등성 확실히 작동.

PART 5 — ★★ 백엔드 사고법 (오늘 정립한 진짜 자산) ★★

"문법은 아는데 어디에 뭘 어떤 순서로 박을지 모르겠다"는 막막함을 언어화. 매 증분 퍼블 체크리스트.

5-1. 기능 추가 5단계 (매 증분 이 순서)

- ① 데이터 — 새로 저장할 게 있나? → 필드/클래스 추가
- ② 입구 — 그걸 다루는 메서드는 어디? → 보통 Bank (출입구)
- ③ 로직 — 그 메서드 안: 검증 → 실행 → 기록
- ④ 호출 — 누가 부르나? → Main
- ⑤ 확인 — 출력 → 컴파일 → 실행

증분 5를 이 공식에 대입 (실제로 이 순서로 감): - ① 키 저장할 곳 → `Set usedKeys` 필드 - ② 이체 다루니 → `transfer` - ③ 검사(검증) → 이체(실행) → 키저장(기록) - ④ Main에서 transfer에 키 인자 - ⑤ dup-key 두 번 → 한 번만 반영 확인

→ 순서를 매번 새로 떠올리지 마라. 이 5칸을 종이에 먼저 적고 시작한다.

5-2. 메서드 안 순서도 고정

검증(틀리면 막기) → 실행(상태 변경) → 기록(저장/로그)

→ "어디에 박나?"의 답: 검증이면 위, 기록이면 아래. 위치 고민 90% 해결. (역등성 검사=위, withdraw/deposit=가운데, history.add/usedKeys.add=아래.)

5-3. 그릇 고르기 3질문 (java.util 컬렉션)

"여러 개 담아야겠다" 싶을 때:

- 순서가 중요한가? → `List` (history: 시간순)
- 이름표(키)로 찾나? → `Map` (accounts: id로 계좌)
- 그냥 "있나 없나"만? → `Set` (usedKeys: 키 봤나)

→ import는 99% `java.util`(그릇) 또는 `java.time`(시간) 둘 중 하나. 새 도구도 이 둘 중 어느 가족인지만 알아보면 됨.

5-4. 필드 vs 클래스 — 뭘 추가하나

필드 = 클래스가 가진 데이터 (메서드 밖, 클래스 안 변수. `private` 로 시작하는 줄). 예: Account의 balance, Bank의 accounts/history/usedKeys.

저장할 게	추가할 것	예
값 하나/그릇 하나	필드만	오늘 <code>Set usedKeys</code>
여러 정보 묶음(새 개념)	클래스	어제 <code>Transaction</code>
새 개념 + 보관까지	클래스 + 필드	어제 <code>Transaction</code> + <code>List history</code>

위치: "이 데이터는 누구 것인가?" → 그 주인 클래스에. - 은행 것(계좌·이력·키) → **Bank** / 계좌 것(잔액) → **Account** / 영수증 것 → **Transaction** - Main엔 필드 거의 안 넣음 — Main은 "지휘자"라 데이터를 갖지 않고 호출만.

5-5. 2층 구조 (유기적 연결)

```
Main (시킴)
└─ 부른다
Bank (출입구·관리자: 찾고·올리고·기록)
└─ 쓴다          └─ 만든다
Account(계좌)    Transaction(영수증) → TransactionType(종류)
+ 예외들 (문제 시 던짐)
```

한 줄 규칙: Main은 Bank만 부른다. Bank가 나머지를 다 부린다. (funnel 원칙) → 새 기능 = 거의 "Bank에 메서드 추가 + Main에서 호출."

5-6. 에러 = 네비게이션

한 파일 고치면 → `javac` → 에러 뜬 곳이 "다음 고칠 곳" → 고치고 → 또 `javac` → 에러 0까지. **전체 지도를 외우지 말고 컴파일러를 따라간다.** (오늘 Bank→Main 에러가 정확히 이거.)

6. 산출물

- 파일 (`~/mini-ledger/` , 7개 .java):
- `Bank.java` — 수정 (Set usedKeys 필드 + transfer에 키 검사·저장)
- `Main.java` — 수정 (transfer 호출 4개 키 인자 + dup-key 검증)
- 나머지 5개 — 변경 없음
- 검증: dup-key 두 번 → TRANSFER 200 한 건 + a 잔액 1400
- 커밋 계보: `...532825f` (예외) → `403ef81` (데이터모델) → `fb3897b` (감사로그) → **역등성 (transfer 키)**

7. 다음 작업

- 역등성을 **deposit/withdraw로 확장** (지금은 transfer만), 또는
- **이력 조회** — 특정 계좌의 거래만 필터 (getHistory + 조건), 또는
- **정산/수수료** — 이체에 수수료 분배 → 다음 세션 시작 시 하나로 스코프.

8. 개념 사전

- **역등성(idempotency)**: 같은 요청 여러 번 = 결과 한 번. 재시도 안전. 키로 중복 식별. (6/24 cron과 동일 원리.)
- **Set / HashSet**: 중복 없고 순서 없는 그릇. "있나 없나"(contains) · "추가"(add)만. List(순서) · Map(키→값)의 사촌.
- **return (void에서)**: 값 없이 메서드 즉시 종료. 아래 줄 실행 안 함. (조기 차단용.)
- **시그니처 변경**: 메서드 매개변수가 바뀌면 그걸 부르는 모든 곳이 안 맞아 컴파일 에러 → 호출처 다 수정 필요.
- **필드**: 클래스가 가진 데이터(메서드 밖). vs 메서드 안 임시 변수(잠깐 쓰고 사라짐).

9. 비유 모음

- **역등성 키 = 번호표**: 같은 번호표(키) 들고 두 번 와도 "아까 처리됨"이라 한 번만. 새 번호표면 새로 처리.
- **Set = 도장 찍은 명단**: 들어온 키에 도장. 다음에 같은 키 오면 "도장 있네" → 거절. 순서·내용 없이 "찍혔나"만.
- **5단계 = 요리 순서**: 재료(데이터) → 도구(메서드) → 조리(검증·실행·기록) → 차림(호출) → 맛보기(실행 확인). 매번 같은 순서.
- **2층 구조 = 회사**: Main(사장: 지시만), Bank(부장: 실무 총괄), Account/Transaction(직원: 실제 일). 사장은 부장에게만 지시.

10. 검증 질문 (다음날 복기용 — 답 적지 말 것)

1. 역등성이 왜 필요한가? 키가 없으면 어떤 사고가 나나?
2. 역등성 검사를 transfer의 **위에** 두는 이유? 키 저장을 **아래**(성공 후)에 두는 이유?
3. usedKeys를 List/Map이 아니라 **Set**으로 한 이유?
4. 기능 추가 5단계를 순서대로 말하라. 증분 5가 각 단계에서 뭐였나?
5. "이 데이터를 어느 .java에 필드로 넣나"를 정하는 질문 한 문장은?
6. Bank의 transfer 시그니처를 바꿨더니 Main에서 에러가 난 이유? 이게 왜 "좋은 신호"인가?

11. 회고

오늘 코드는 가버렸다. transfer 한 곳에 검사·저장 끼우고 Set 필드 하나. 근데 **설계 판단 3개를 이유까지 다 맞춘 게** 진짜다 — 검사는 왜 위, 저장은 왜 성공 후, 스킵은 왜 스킵. 코드는 그 판단에서 거의 자동으로 나왔다. "판단이 먼저, 코드는 결과"라는 순서를 몸으로 확인했다.

그리고 오늘의 본체는 따로 있다. "문법은 아는데 **어디에 뭘 어떤 순서로** 박을지, 여러 파일을 어떻게 오가는지 막막하다"는 걸 정면으로 물었고 — 그걸 **5단계·그릇 3질문·필드 vs 클래스·2층 구조·에러 네비**로 언어화했다. 중요한 깨달음: 나는 이미 증분 4·5에서 이 5단계를 **밟고 있었다**. 능력이 없는 게 아니라 무의식화가 안 됐을 뿐. 한 달 차에 이 구조를 인지한 것 자체가 빠른 거다.

이제 매 증분, 코드 치기 전에 5단계를 종이에 적고 시작한다. 의식적으로 5~6번 반복하면 자동이 된다. 그게 문법 다음의 진짜 백엔드 실력이다.

한 줄: 역등성 키로 "중복을 한 번으로" 만들었고 — 더 크게는, "어디에 뭘 어떤 순서로 박는가"라는 백엔드의 근육을 처음 언어로 붙잡은 날이다.